

Practical 6 (References)



Steps to Use Preferences

- To add preferences to the application, the following steps are required:
 - Retrieve an instance of a **SharedPreferences** objects
 - Create a **SharedPreferences.Editor** to modify preference content
 - Make changes to the preferences using the Editor (add, update and delete)
 - Commit your changes

Creating Preferences

- Preferences can be private or shared
- The private preferences are restricted for the private use of the Activity:

```
SharedPreferences prefPrivate =  
    getPreferences (MODE_PRIVATE) ;
```

- The shared preferences could be used in any activity of the application through the preference name:

```
SharedPreferences pref =  
    getSharedPreferences (  
        "MySharedPreferences", 0) ;
```

Reading Preferences

- The preferences could be read from the **SharedPreferences** objects
- Each preferences could be accessed by its name and provided with a default value if not found
- The preferences are accessed by different data type
- Some methods are available for easy access of data from the preferences

SharedPreferences Methods

- **getBoolean()** – retrieves a specific Boolean-type preference by name
- **getFloat()** – retrieves a specific Float-type preference by name
- **getInt()** – retrieves a specific Integer-type preference by name
- **getLong()** – retrieves a specific Long-type preference by name
- **getString()** – retrieves a specific String-type preference by name

Altering Preferences

- One can add, update or delete preferences
- To make changes, a preference **Editor** is required from **SharedPreferences.Editor**
- The Editor is created for a created **SharedPreferences** object

```
SharedPreferences.Editor prefEditor =  
    pref.edit();
```

Editor Methods

- **putBoolean()** – sets a specific Boolean-type preference by name
- **putFloat()** – sets a specific Float-type preference by name
- **putInt()** – sets a specific Integer-type preference by name
- **putLong()** – sets a specific Long-type preference by name
- **putString()** – sets a specific String-type preference by name

Using Editor

- After creating the **Editor** from existing **SharedPreferences**
- Multiple action of including data (add or update) could be executed
- The changes will only take place when the editing is committed using **commit()**

- Example:

```
prefEditor.putLong("SomeLongPref", 0);  
prefEditor.commit();
```

Storing in Files

- Android application data are stored on the Android file system in the following directory:
`/data/data/<package name>/`
- As you had noticed by now, some default subdirectories are used for storing database and preferences as well as files
- You can also create custom directories when the need arise



File & Directory Management

- Some **Context** (**android.context.Context**) methods for file and directory management as follow:
- **openFileInput()** – opens an application file for reading (**/files**)
- **openFileOutput()** – opens an application file for writing (**/files**)
- **deleteFile()** – deletes an application file by name (**/files**)

Write File – example

```
FileOutputStream fos;  
String strFileContents = "Let's write this in  
the file. ";  
  
try {  
    fos = openFileOutput("Testfile.txt",  
        MODE_PRIVATE);  
    fos.write(strFileContents.getBytes());  
    fos.close();  
} catch (Exception e) {  
    e.printStackTrace();  
}
```

Append File – example

```
FileOutputStream fos;  
String strFileContents = "Let's add this in the  
file. ";  
  
try {  
    fos = openFileOutput("Testfile.txt",  
        MODE_APPEND);  
    fos.write(strFileContents.getBytes());  
    fos.close();  
} catch (Exception e) {  
    e.printStackTrace();  
}
```

Reading File – example

- Example to open the file previous created and written with some data:

```
import java.io.FileInputStream;
```

```
:
```

```
String strFileName = "Testfile.txt";
```

```
FileInputStream fis;
```

```
StringBuffer buffer = new StringBuffer();
```

```
DataInputStream dataIO;
```

```
String strLine = null;
```

Reading File – example (cont.)

```
try {  
    fis = openFileInput(strFileName);  
    dataIO = new DataInputStream(fis);  
  
    while ((strLine = dataIO.readLine()) != null) {  
        buffer.append(strLine + "\n");  
    }  
  
    dataIO.close();  
    fis.close();  
} catch (Exception e) {  
    e.printStackTrace();  
}
```